When More Parallelism Means More Work

Chess Search with MPI, OpenMP, and GPU Offload

Jia Siqi (A0333724H) Jia Zeyue (A0326500U)

DSA5208 Project 3 9 November 2026

Abstract

Parallel chess search creates a conflict between exposing work and avoiding it: independent subtrees are easy to distribute, but they cannot exploit the same alpha–beta bounds as a sequential search. We implement serial negamax, MPI+OpenMP frontier decomposition, a seeded-root alternative, and two OpenMP GPU offload strategies. On NUS Vanda, six additional positions yield 120 complete three-trial CPU configuration groups, including 24 strong-scaling points from 72 to 720 allocated cores. At 720 cores, the recorded search-region medians for four longer workloads are 5.57–21.84 times as fast as serial alpha–beta, while two shallow middlegames remain slower. These are allocation-dependent observations, not population estimates or end-to-end speedups. A pair of baselines separates scaling of a fixed task graph from acceleration of the original algorithm. On the initial position, 112.95-fold fixed-task speedup becomes only 3.04-fold speedup over serial alpha–beta because decomposition expands search work by 36.3 times. Seeded-root search reduces wasted work but exposes load imbalance; deeper splitting helps large allocations while often hurting a 16-core run. GPU chunking reduces speculative tree construction, yet remains slower than the paired serial CPU search. Explicit profiling shows that raw data copies account for only 2.9% of a depth-eight GPU run: CPU tree generation and the combined launch/search phase dominate. These observations motivate workload-aware splitting and bound-preserving scheduling; their benefits require controlled follow-up rather than a universal preference for more cores.

1 Problem, scope, and research questions

A chess engine chooses a move by considering its own continuations and its opponent's best replies. A position contains the pieces, side to move, castling rights, en-passant square and move counters. Legal moves are edges of a game tree, whose shape changes with checks, captures and terminal outcomes. One *ply* is one player's move; depth eight is eight plies, not eight full turns.

Our task is to compute a deterministic fixed-depth score and a maximizing legal root move. This isolates search and scheduling from playing-strength engineering. Leaves use material values of 100, 320, 330, 500 and 900 for pawn, knight, bishop, rook and queen. Scores are in centipawns from the root side's perspective. Checkmate receives a large score adjusted for distance; stalemate and supported draw conditions receive zero. There is no learned evaluation, transposition table, quiescence search, iterative deepening or heuristic move ordering. A reported move is best under this model and depth; it need not be a strong move according to a competitive chess engine.

The study asks three questions. First, how does frontier depth trade parallel capacity against lost pruning? Second, does preserving an initial root bound improve runtime when static MPI assignment remains unbalanced? Third, can GPU offload repay CPU preparation and device execution costs for this irregular search? We evaluate multiple positions to identify both useful scaling and small-workload failure cases. The positions are fixed benchmarks, including synthetic states, rather than a representative sample of human games.

2 Algorithm and sources of parallel dependence

2.1 From a chess position to a search tree

The input is one board state, not a dataset of games. At the root, the side to move chooses a legal move. The opponent then chooses a reply, followed by our next choice, and so on. These alternating choices form a tree: a node is a position and an edge is one legal move. The number of children varies because different positions allow different moves; checkmate, stalemate and draws can stop a branch early. Appendix A shows the seven exact input boards alongside their FEN encodings.

At a fixed depth, the program does not play out every game to its final result. It scores the reached leaves using the stated material values and chooses a root move assuming that the opponent will choose the reply least favorable to us. Thus the first two levels are "choose the best move, then face the worst reply". More plies repeat the same rule. This is a deterministic benchmark of search and parallel scheduling, rather than an attempt to predict human play.

2.2 Negamax and alpha–beta pruning

For legal moves $M(s)$ and the state s_m after move m, negamax writes minimax as

$$V(s,d) = \begin{cases} E(s), & d = 0 \text{ or } s \text{ is terminal}, \\ \max_{m \in M(s)} [-V(s_m, d-1)], & \text{otherwise}. \end{cases} \tag{1}$$

Terminal rules are applied before the material cutoff. The implementation negates the evaluation perspective after each move and reverses board changes when returning. An exhaustive minimax backend disables pruning and provides a shallow search-control reference.

Alpha–beta carries bounds (α, β) and searches a child with $(-\beta, -\alpha)$. Once $\alpha \geq \beta$, further continuations cannot change the relevant decision and may be omitted [1]. The following small two-ply example gives the intuition; scores are from the root player's view:

Our candidate move	Opponent replies examined	What we know
A	+100, +200	Opponent chooses +100
B	0, then replies not yet examined	B can yield at most 0

After completing A, the root has a move worth +100. The first reply to B already proves that a minimizing opponent can hold B to 0 or less, so no remaining reply to B can make it the better root move. This is a *cutoff*: the skipped branches have no effect on the best root decision. Searching A and B independently in parallel may inspect those branches before A's bound is known. Move order therefore changes work even though it does not change the exact fixed-depth minimax value.

This dependency makes the algorithm a meaningful parallelization problem. Within a subtree, recursive choices and cutoffs remain dependent; across subtrees, a shared search bound could eliminate work. Our basic parallel implementation deliberately removes some of that dependency to expose tasks. It must then pay for the resulting additional node visits and merge exact scores. Running unrelated chess positions on different nodes would not expose this particular tradeoff.

2.3 Implementation boundary

Project C++17 code implements search, evaluation, frontier construction, MPI/OpenMP scheduling, GPU traversal and instrumentation. The unmodified MIT-licensed Disservin `chess-library`, revision `2a608d58f44776909bb6e2c09505e97ba1451319`, supplies board state, FEN parsing, legal moves, move application/reversal, draw helpers and UCI formatting [3]. Its license and provenance are included. All calls to that host-oriented library execute on the CPU. It does not supply our search or parallel scheduler. The documented instructor guidance accepted this credited dependency.

3 Parallel designs and correctness arguments

3.1 Root splitting and deeper frontiers

The basic hybrid design assigns legal root moves to ranks by task index modulo rank count. OpenMP uses a dynamic loop within each rank. Every task owns its board and counters and searches with a full window. MPI combines scores with `MPI_Allreduce`; unowned scores use a negative-infinity sentinel. MPI calls occur on the main thread, using `MPI_THREAD_FUNNELED`.

A root often supplies only tens of tasks. To use hundreds of workers, split depth $k = 2$ or 3 constructs a larger CPU frontier. Each frontier board is searched independently for the remaining depth; exact results are combined bottom-up with the negamax recurrence, including the appropriate sign change at every ply. Correctness follows by induction: a frontier search returns the exact value of its remaining subtree, and maximizing negated exact child values recovers the parent value. Early terminal nodes remain frontier leaves. This argument relies on full-window task values; merging arbitrary cutoff bounds as exact scores would be incorrect.

Ranks currently build the same frontier and each scans the task loop, skipping unassigned indices. MPI ownership is static; OpenMP dynamic scheduling cannot move an expensive task to another rank. Deeper frontiers increase board-storage and construction costs, replicated rank-local work, reductions and merge work, as well as the number of available tasks. The node counter counts frontier construction once logically, not once per rank; it consequently understates physical duplicated preparation work at larger rank counts.

3.2 Seeded-root search

The `mpi-pv` and `omp-pv` backends first search the first legal root move exactly. Its score a is broadcast and used as a fixed lower bound for the remaining root choices. In the negated child search, the window is $(-\infty, -a)$. If a child search cuts off, its root-side result cannot improve a; if it proves an improvement, the required value is recovered. Keeping the exact seed and taking the root maximum thus preserves the root value, although individual rejected branches may return bounds rather than exact scores. Stable root ordering resolves equal scores.

This is inspired by the principle of searching a first child before releasing siblings, as in Young Brothers Wait [2]. Our implementation is a *root-only fixed-bound variant*; it is not a complete recursive YBWC scheduler. It neither refreshes a after each improvement nor dynamically balances work between MPI ranks. These restrictions let us examine the benefits of retaining pruning separately from scheduling quality.

3.3 OpenMP GPU offload

The first GPU design expands the entire legal tree on the CPU, packs flat node and leaf arrays, then uses `target teams distribute parallel for` over root subtrees. Each device worker performs explicit-stack alpha–beta traversal and leaf evaluation. Device-side pruning cannot undo CPU work already spent constructing the full tree.

The chunked design preserves CPU alpha–beta above a fixed chunk depth. When a chunk is reached, the CPU expands that local tree and sends its child branches to the device with independent full windows. Exact child results are merged on the CPU and search continues. This reduces speculative construction but increases the number of target launches. Chunk depth controls the balance between fewer, larger transfers/launches and extra generated nodes. Neither backend is a multi-GPU MPI implementation: MPI+OpenMP CPU search is the principal hybrid solution, and GPU offload is an additional design comparison.

Both designs preserve the same evaluation and terminal semantics. Actual A40 runs require `OMP_TARGET_OFFLOAD=MANDATORY` and check `omp_is_initial_device()` to reject silent CPU fallback. Profiling variants use explicit allocation and `omp_target_memcpy` to distinguish copies from

target-region execution [5].

4 Experimental protocol and metrics

4.1 Vanda workloads and resource layouts

CPU experiments ran on NUS Vanda using PBS, GCC 14.2.0, Open MPI 5.0.7, C++17, `-O2` and OpenMP. The 53 captured multi-position session logs identify the launch node CPU as Intel Xeon Platinum 8452Y, 72 visible CPUs, two sockets and one hardware thread per visible core. This is not independent verification of every remote rank's CPU or clock. Hostnames, code hashes and allocation details are retained to disclose differences between jobs.

The original initial-position depth-eight experiment is extended with six positions (Table 1). Strategy comparisons use 1/2/4 nodes and four threads per rank, testing split depths 1/2/3 and seeded root. Scaling uses 1/2/4/10 nodes, one rank per node and 72 requested threads per rank, at fixed split depth three. Each new position also has three-trial serial and one-worker task baselines. There are 120 complete configuration groups, including 24 full-node scaling points, and 360 successful formal runs. The 384 archived run records also include pilots and one unsuccessful formal timeout.

Table 1: Additional workloads. Depth is fixed within each position's curve. The two depth-four middlegames are explicitly retained as small-workload cases.

Position	Depth	Serial score	Interpretation
Kiwipete	6	−90	Move-rich middlegame
Rook/pawn	9	100	Reduced-piece endgame
Tactical	6	−660	Tactical benchmark state
Minor pieces	9	190	Synthetic bishop/knight/pawn endgame
Ruy Lopez	4	−100	Short middlegame workload
Queen's Gambit	4	−100	Short middlegame workload

Pilots increased depth to obtain measurable serial work. For the two shallow middlegames, depth-five attempts exceeded an outer launcher-inclusive timeout; the last successful depth, four, was used. The failed logs contain launcher messages but no search result, so they do not establish that chess search itself needed 45 seconds or that depth five was computationally infeasible. This is a workload-selection limitation, not evidence that those chess positions are inherently easy. Their millisecond runtimes are unsuitable for strong throughput claims. New CPU runs used source revision `237f06f`; the original scaling source was `efef92e`. Their serial and deeper-frontier implementations are unchanged. Newer GPU source hashes differ and are recorded separately. The artifact archive pins the measured sources instead of assuming all results came from the current repository head.

All corrected launches use `-bind-to none`. An earlier controlled four-thread experiment on the same node took 6.229 seconds under default one-core binding versus 2.432 seconds unbound, with identical search work. A separate affinity probe confirmed that the unbound rank remained within the PBS allocation on the tested node. Open MPI explicitly recommends either no binding or enough processing elements for threaded processes [4]. The measured affinity correction addresses that particular confinement; `-bind-to none` alone is not a verified placement policy for every run. It does not optimize NUMA locality; thread placement and per-rank memory access remain uncontrolled factors.

4.2 Timing, repetition, and validation

We report medians of three fresh process invocations in each allocation, without removing a warmup. Observed ranges show repeat variability, not confidence intervals. Three trials on one

allocation do not quantify variability across independent allocations. The older initial-position one-worker task baseline has only one trial; its derived efficiencies carry that limitation.

The program's MPI time is the maximum rank search-region time: frontier construction, local search, score/counter reductions and merging are included; process startup and the final timing reduction are excluded. There is no explicit pre-search MPI barrier in this path; the maximum of rank-local durations is therefore not a synchronized global start-to-finish measurement. The added Python runner separately records walltime around the launcher as `launch_seconds`. GPU total time includes preparation and offload. The original implicit-map target timer includes copies and synchronization; explicit-copy profiling separates H2D and D2H, but its target-search region still includes launch and synchronization rather than pure device arithmetic.

Every complete new CPU group has three distinct successful trials whose scores match the corresponding serial reference. Shallow tests recorded in the project compare alpha–beta with exhaustive minimax across seven FENs and depths 1–3, including checkmate, stalemate, promotion and en passant. Target-host checks exercise device traversal logic on the CPU; real GPU logs additionally show `ran_on_device=1`. Agreement under shared chess rules validates the search-control paths; it is not an independent proof of all chess rules. Equal-score legal moves may differ, so score agreement is the automated archive criterion. It should not be confused with independent verification of each chosen move's optimal value in every deep run.

4.3 Two baselines and a work-inflation model

Let T_s be serial alpha–beta time, $T_{k,1}$ one-worker time for the same split-depth-k task algorithm and $T_{k,p}$ its time on p allocated cores. We distinguish

$$S_{\mathrm{alg}}(p) = T_s/T_{k,p}, \qquad S_{\mathrm{task}}(p) = T_{k,1}/T_{k,p}, \qquad E_{\mathrm{task}}(p) = S_{\mathrm{task}}(p)/p. \tag{2}$$

Define time inflation $H_k = T_{k,1}/T_s$ and node inflation $A_k = N_k/N_s$. They are different quantities: node costs, scheduling, construction and hardware differences prevent treating node visits as a uniform unit of time. By definition,

$$\boxed{S_{\mathrm{alg}}(p) = \frac{pE_{\mathrm{task}}(p)}{H_k}}. \tag{3}$$

Thus useful acceleration over serial requires $E_{\mathrm{task}} > H_k/p$. This identity is an accounting device, not a fitted performance law. It explains why impressive fixed-task speedup can coexist with modest algorithm speedup. We additionally use $pT_{k,p}/T_s = p/S_{\mathrm{alg}}$ as a rough allocated core-time ratio. It is not measured energy or billing cost, and baseline hardware differences limit causal interpretation.

5 CPU results: measured gains and limits

5.1 Reinterpreting the original initial-position result

At depth eight, serial alpha–beta visits 25,604,258 nodes in a median 5.34012 seconds. Split depth three visits 929,868,807 nodes: $A_3 = 36.32$. Its one-worker time is 198.129 seconds ($H_3 = 37.10$); the 720-core median is 1.75406 seconds. Consequently $S_{\mathrm{task}} = 112.95$ and $E_{\mathrm{task}} = 15.7\%$, but $S_{\mathrm{alg}} = 3.04$. Equation 3 explains the difference without selecting a convenient baseline. An earlier feasibility job had 53 seconds of PBS walltime despite 1.593 seconds of internal search, illustrating the importance of scope.

The 72/144/288/720-core medians were 9.094/3.048/1.945/1.754 seconds. The 72-to-144-core ratio exceeds two, as do some new workload segments. We report these observations without attributing them to caching or scheduling: rank assignment changes, NUMA behavior, processor frequency and allocation effects were not isolated. Strong scaling of an application region does not establish an end-to-end superlinear algorithmic benefit.

5.2 Six additional positions

Table 2 and Figure 1 show that neither uniformly good nor uniformly poor scaling describes these workloads. At 720 cores, Kiwipete, rook/pawn, tactical and minor-piece positions are 10.31, 5.57, 11.84 and 21.84 times as fast as their serial references. All four improve in median search time as resources increase through the measured configurations. The minor-piece case has substantially lower split-three work inflation than Kiwipete (Table 4); less destroyed pruning is one plausible contributor to its higher algorithm speedup, but the observations do not isolate it from task balance or hardware effects.

Table 2: Median search times (seconds), split depth three. S_{alg} and E_{task} are for 720 cores. Each new entry uses three trials.

Position	d	Serial	72	144	288	720	S_{alg}	E_{task} (%)
Kiwipete	6	5.142	7.153	2.351	0.978	0.499	10.31	52.6
Rook/pawn	9	3.188	3.432	1.196	0.831	0.572	5.57	14.5
Tactical	6	1.295	0.905	0.375	0.172	0.109	11.84	29.4
Minor pieces	9	10.557	3.789	1.284	0.647	0.483	21.84	20.4
Ruy Lopez	4	0.006	0.027	0.046	0.058	0.067	0.09	0.4
Queen's Gambit	4	0.022	0.088	0.077	0.072	0.080	0.28	1.2

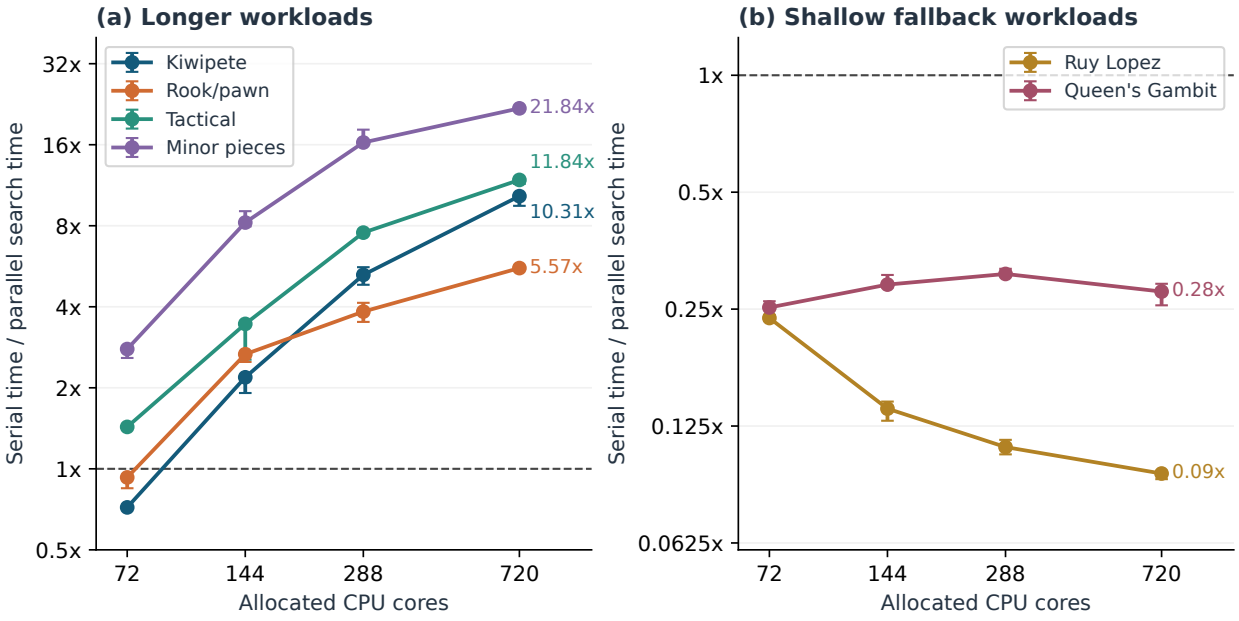


Figure 1: Search-region speedup with common axes within each workload group. Panel (a) contains four longer workloads; panel (b) contains two shallow fallback workloads and uses a separately labeled range. Dashed lines mark serial parity. Whiskers span $[T_{s,\min}/T_{p,\max}, T_{s,\max}/T_{p,\min}]$, so both baseline and parallel observed variation are included. These are unpaired descriptive extreme-ratio envelopes from three trials, not confidence or prediction intervals. The final point is labeled with its ratio of medians.

The two shallow middlegames show the opposite regime. Ruy Lopez slows as cores increase, while Queen's Gambit stays near 0.07–0.09 seconds and remains slower than its 0.022-second serial run. With depth four and split depth three, each ordinary frontier task has at most one ply left; in the logged full-frontier runs these positions record zero cutoffs. The observed slowdown is consistent with inadequate work per task relative to setup and synchronization. However, CPU phase timing was not collected, so these data do not identify construction, collectives or another component as the dominant cause. A larger-task controlled sweep would be needed to test that

explanation. The correct conclusion is about these small workloads, not about the scalability of all positions arising from those openings.

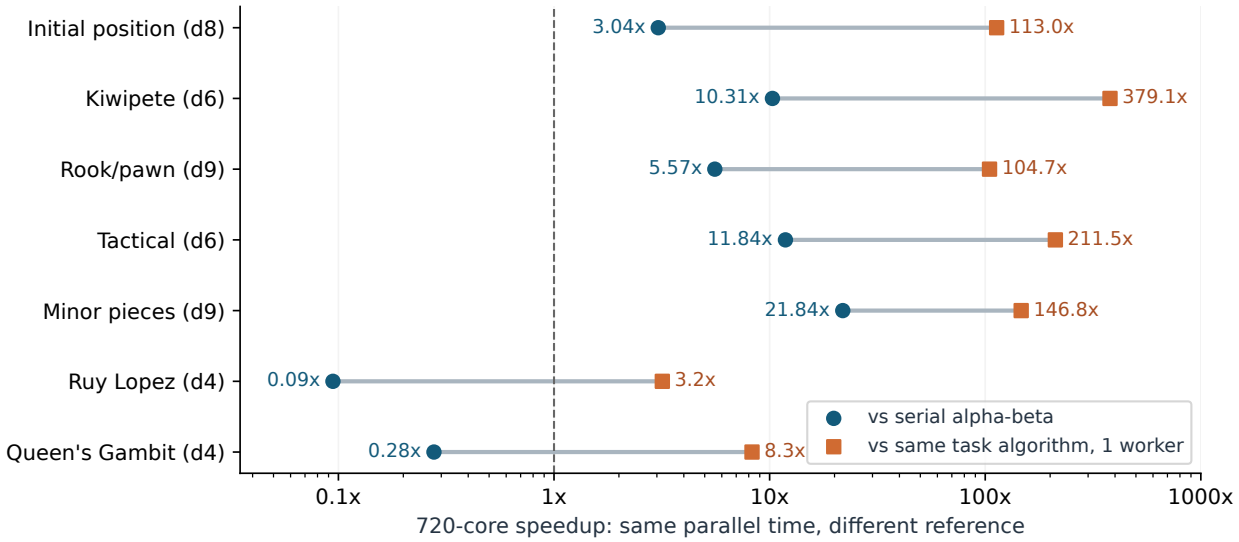


Figure 2: Two reference times divided by the same 720-core parallel search time. A blue circle uses serial alpha–beta; an orange square uses a one-worker run of the same split-three task algorithm. Each connector changes only the reference-time numerator; the parallel-time denominator and execution remain fixed. Points are ratios of medians; the original initial position's task baseline has one trial. A task speedup above one can coexist with a slowdown versus serial (bottom two rows).

5.3 Efficiency and resource cost

At 720 cores, Kiwipete has 52.6% fixed-task efficiency and 10.31-fold algorithm speedup; minor pieces has 20.4% efficiency but 21.84-fold algorithm speedup. The ordering by task efficiency differs from the ordering by speedup over each position's own serial reference; it is not a direct comparison of absolute usefulness across different chess workloads. Their time-inflation factors are about 36.78 and 6.72, respectively. Holding a measured parallel time fixed, a larger one-worker reference time in the numerator raises the reported task speedup by definition. The reference choice itself does not improve execution time. This arithmetic relation cannot diagnose the cause of a measured slowdown.

The longer workloads still consume roughly 33–129 times the serial *allocated search-region core-time* at 720 cores. More cores buy latency, not automatically economical resource use. For the minor-piece case, going from 288 to 720 cores reduces median time from 0.647 to 0.483 seconds, while increasing core-time from about 186 to 348 core-seconds. This gives a concrete basis for selecting a configuration by either latency or resource budget. It does not include launcher costs or measure energy.

5.4 Search time is not launcher-inclusive response time

The added runner also records elapsed time around each `mpirun` invocation. Using those recorded medians changes the speedup substantially (Table 3). Both the serial and parallel paths use the MPI executable and launcher; the serial-wrapper measurement is not a standalone serial program's response time. Startup, shell/network delays, MPI initialization, output and other non-search work remain in this measure. The difference from search time cannot be assigned solely to communication overhead, and the difference between medians is not a median per-run overhead. For example,

Kiwipete's launcher-inclusive serial median is 19.462 seconds although its serial search median is 5.142 seconds. The wrapper noise is substantial and cannot be ignored or cleanly subtracted.

Table 3: Ratios of medians under two timing scopes, using three invocations per configuration. Wrapper time is a directly measured elapsed time, but the serial wrapper includes MPI launch and allocation effects. It is not a clean hardware-independent application response benchmark.

Position	Serial wrapper (s)	720-core wrapper (s)	Search ratio	Wrapper ratio
Kiwipete	19.462	2.136	10.31	9.11
Rook/pawn	4.844	2.376	5.57	2.04
Tactical	2.932	1.982	11.84	1.48
Minor pieces	11.798	2.230	21.84	5.29
Ruy Lopez	1.171	2.787	0.09	0.42
Queen's Gambit	1.777	2.135	0.28	0.83

The minor-piece speedup falls from 21.84 in the search region to 5.29 in the wrapper record; tactical falls from 11.84 to 1.48. We therefore restrict the main claim to measured search-region acceleration. A production latency claim would require a controlled end-to-end protocol, for example persistent workers or matched process-start conditions, with more independent allocations.

5.5 What deeper splitting actually costs

Table 4: Search-work inflation versus serial. N_s is in millions; $A_k = N_k/N_s$ uses node counts, while H_3 uses measured one-worker times.

Position	N_s	A_1	A_2	A_3	H_3
Kiwipete	19.227	2.65	8.13	38.55	36.78
Rook/pawn	11.798	3.31	13.14	29.77	18.80
Tactical	3.200	3.10	6.17	27.36	17.86
Minor pieces	66.944	1.66	4.24	6.60	6.72
Ruy Lopez	0.024	2.72	10.20	32.80	33.71
Queen's Gambit	0.060	5.22	10.48	28.71	29.87

Kiwipete's split-three frontier has 99,950 stored frontier nodes, compared with 9,323 in the original initial position. It offers many tasks but increases search work to 741 million node visits, 38.55 times serial. The minor-piece case has only 3,566 stored frontier nodes and 6.60-fold node inflation. Stored frontier nodes include internal frontier structure; this is not exactly the number of leaf tasks. Task counts and task sizes both matter. A count-only argument that there are "enough tasks for 720 cores" cannot establish balance, pruning efficiency or useful speedup.

At 16 allocated cores, no position's split-three configuration is the fastest of the four compared designs (Table 5). On Kiwipete and the minor-piece workload, split two has the lowest recorded median; seeded root has the lowest median on rook/pawn and Queen's Gambit; split two has the lowest median on tactical, while root splitting has the lowest median on Ruy Lopez. The small-case rankings require caution because millisecond differences and serial-baseline allocation differences can dominate. Within each new position/rank-count job, the strategy variants share an allocation, but run in a fixed, non-randomized order. This controls host identity within that job, not order effects, clocks or baseline placement. Nevertheless, the substantial Kiwipete gap—3.282 seconds for split two versus 13.008 for split three—is evidence against selecting a split depth solely because it fills a large allocation. It does not establish a universally optimal split-depth policy.

Table 5: Four nodes, one rank and four threads per node (16 cores). Times in seconds; bold is the smallest observed median within the row, not a statistical claim of superiority. Same depth and position are retained across methods.

Position	Root	Split 2	Split 3	Seeded root
Kiwipete	5.5134	**3.2820**	13.0077	4.7151
Rook/pawn	1.1277	3.3670	4.4989	**1.0286**
Tactical	1.1239	**0.5567**	2.2747	1.2260
Minor pieces	9.9059	**3.4022**	4.5558	10.8523
Ruy Lopez	**0.0180**	0.0245	0.0561	0.0206
Queen's Gambit	0.0269	0.0350	0.0967	**0.0256**

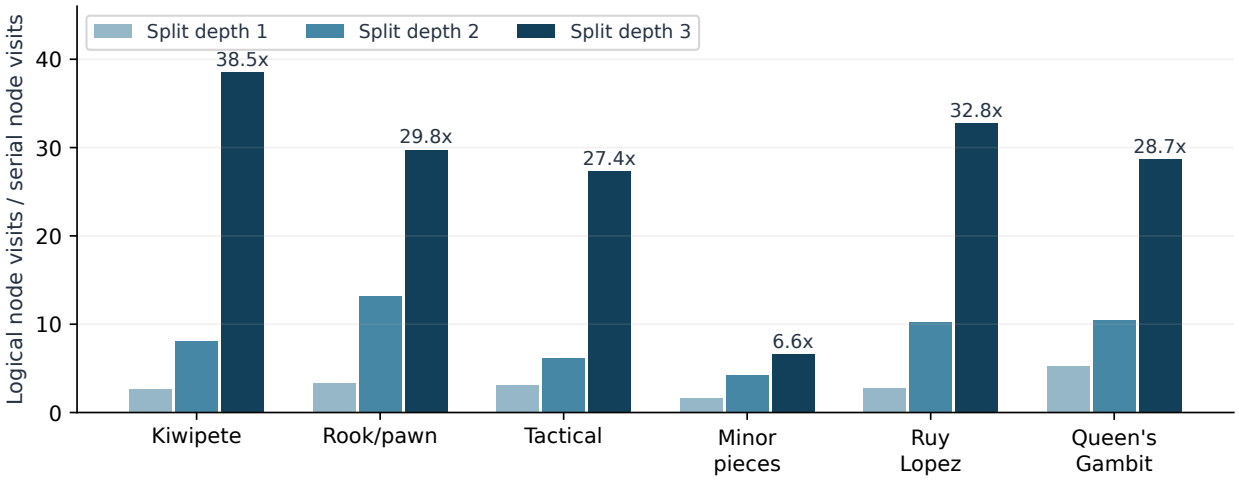


Figure 3: Logical search-work inflation from splitting. All positions share one vertical scale. Dark bars show split depth three; labels give its node ratio. Counts are constant across the recorded ordinary-task configurations for a given position and split depth. Replicated rank-local frontier preparation is counted once logically, so this is not total physical CPU work or elapsed overhead.

5.6 Preserving pruning does not resolve load imbalance

The original seeded-root initial-position study visits exactly the same 25,604,258 nodes as serial at depth eight, yet medians on one/two/four four-thread nodes are 3.010/2.853/4.205 seconds. An instrumented four-node run attributes only 0.347 seconds to the seed. Afterwards rank zero searches 14.14 million nodes for 3.459 seconds, whereas other ranks search 2.64–3.88 million nodes in 0.333–0.444 seconds. The longest observed rank search is approximately three times the mean of the four rank work times. These timers cover local search regions rather than a measured idle-time trace, but directly expose a severe static-assignment imbalance. The seed alone is not an adequate explanation of the slow critical path.

In the new rook/pawn workload, seeded root improves from 1.448 seconds on one four-thread node to 1.029 on four, compared with 3.188 seconds serial. In the minor-piece workload, it remains near 11 seconds despite more nodes. A good bound and a balanced schedule are distinct requirements. The initial legal move is not guaranteed to be a good seed; the fixed bound also misses later improvements available to serial alpha–beta. We therefore avoid claiming a universal seeded-root advantage or equating this implementation with a mature parallel chess engine.

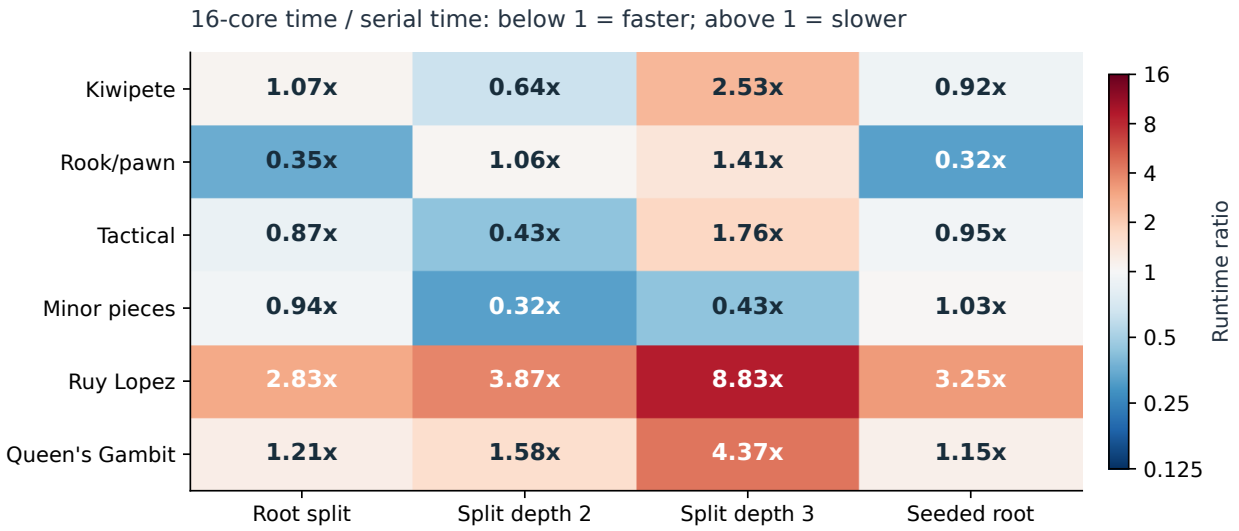


Figure 4: At 16 allocated cores, numbers show each method's median search time divided by the same position's serial median. Blue means faster than serial, red slower; one is parity. The color scale covers all displayed values. Columns are tested strategies rather than a claim of exhaustive optimization. No statistical significance is established by the smallest cell in a row.

6 GPU results: what phase timings establish

6.1 Full trees versus on-demand chunks

On an A40 node with NVHPC 24.7 and the GCC 13.2 toolchain, the full-tree backend returns serial-matching scores at depths three through five. Its CPU-built tree grows from 9,322 to 5,072,212 nodes; at depth five its initial pilot total is 1.798 seconds versus 0.007303 seconds for paired serial search. That pilot establishes correct offload, not useful acceleration.

A later same-allocation comparison repeats each design three times (Table 6). Chunk depth three generates more nodes than chunk depth two, but reduces chunk launches from 844 to 58 and is faster. Thus fewer generated nodes alone do not minimize GPU runtime; launch granularity also matters. Relative to full-tree offload, chunk depth three reduces the measured median by 3.88 times, yet remains about 59.6 times slower than paired serial. This is a reduction within the measured offload family. It does not establish a GPU advantage over the CPU reference, nor isolate launch count as its cause: CPU generated work, transfers and device traversal also change with chunk depth.

Table 6: Initial position, depth five, same A40 allocation, medians of three trials. Generated chunk nodes are cumulative, not peak memory footprint.

Method	Generated nodes	Chunk launches	Total (s)	Target (s)
Serial CPU	—	—	0.007177	—
Full-tree GPU	5,072,212	1	1.656630	0.560216
Chunks, depth 2	431,665	844	0.501165	0.419824
Chunks, depth 3	615,039	58	0.427455	0.310383

At depth eight, one chunk-depth-three pilot takes 154.521 seconds, versus 3.940 seconds for its paired serial search, generating 428.8 million nodes across 30,619 chunks. Deeper search amortizes one-time setup but exposes the large cumulative cost of repeated construction and target work. We do not combine GPU and CPU medians from different compiler paths as if hardware were the only changed variable.

6.2 Explicit profiling challenges the transfer-only explanation

The explicit-copy depth-eight profile takes 154.620 seconds, with 57.525 seconds CPU preparation, 3.563 H2D, 0.853 D2H, 0.243 device probe and 92.313 seconds in target-search regions. Copies total 4.417 seconds, or 2.86% of end-to-end time; CPU preparation is 37.20%, and target launch/search is 59.70%. Allocation and residual bookkeeping account for the remainder. The run uploads 33.54 GB cumulatively, but volume alone does not identify the dominant cost. Reducing that measured copy time to zero while leaving other phases unchanged would yield only

$$\frac{154.620}{154.620 - 4.416629} \approx 1.029 \tag{4}$$

fold total speedup. This counterfactual holds the recorded non-copy phases fixed. It is not a bound on a redesigned implementation that also changes computation, overlap or CPU preparation, and the single profile does not estimate run-to-run uncertainty. Even eliminating the measured CPU preparation alone would leave roughly 97 seconds, far above the paired 3.689-second serial search in this profiling job.

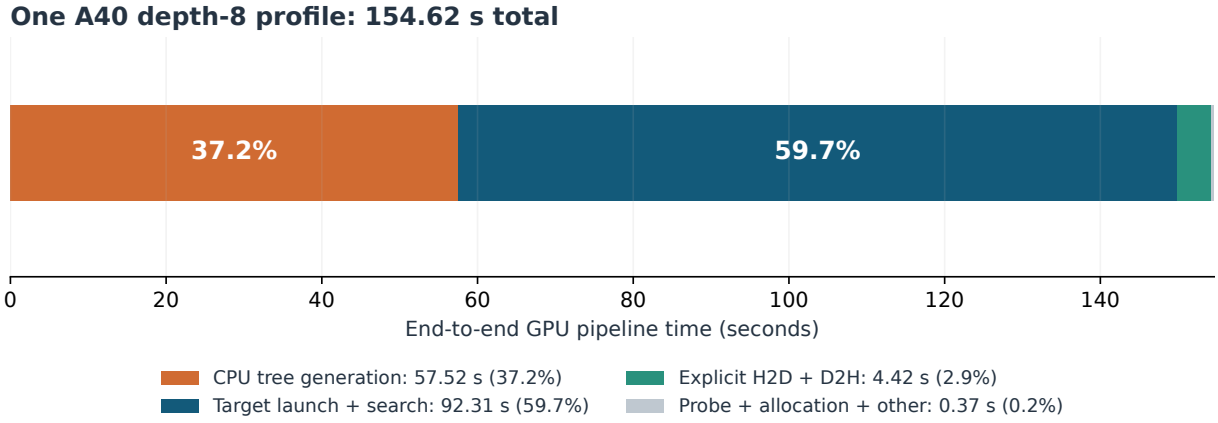


Figure 5: One explicit-copy A40 depth-eight profile, partitioning its measured 154.620-second total. Large segments are labeled with fractions of total; the legend preserves small phases instead of visually hiding them. “Other” includes the device probe, allocation and residual host work. Target time includes launch and synchronization; it is not a pure arithmetic kernel timer. These single-run fractions do not prove a hardware-general bottleneck.

The correct next target is therefore the search representation and granularity, not only raw transfer bandwidth. CPU-native move generation forces speculative preparation, while branch divergence, limited branch-level parallelism and many launches are plausible contributors to target-region time. These latter mechanisms were not separately profiled, so the data do not establish their individual contributions. Profiling has narrowed the design question without answering it completely. Persistent device data, GPU-native move generation or larger batched frontiers would change the design and require new validation; they are future work, not demonstrated optimizations.

7 Limitations, design lessons, and reproducibility

7.1 What the evidence can and cannot support

First, these are fixed-position strong-scaling experiments. Depth and position are held fixed within each curve; depths differ across workloads. Increasing chess depth is not a weak-scaling protocol

because branching and pruning change nonlinearly. We make no weak-scaling claim and do not infer population-wide chess performance from six selected additional positions.

Second, the source and numerical workload are controlled more tightly than the physical execution. Jobs use different allocations; the logs do not establish uniform clocks, NUMA locality or background load. New CPU runs use three repetitions in one allocation, not three independently randomized allocations. Displayed ranges are descriptive envelopes, not inferential intervals. Claims of superlinear speedup, small performance differences or universal strategy superiority would exceed this evidence.

The workload groups also differ in position and depth; duration is not an independently manipulated factor. The shorter cases cannot establish a causal work-size threshold. A same-position depth sweep would be required. All full-node runs fix one rank per node, so this study cannot identify an optimal rank/thread layout or separately quantify an MPI tax against a standalone OpenMP build.

Third, serial baselines and MPI allocations differ in resource environment. The one-worker task baseline includes the hybrid path's bookkeeping; node inflation, time inflation and core-time are complementary measurements, not interchangeable. Frontier construction is replicated at each rank but counted once logically. Timing scopes must remain explicit, especially for subsecond searches and GPU target timers.

Fourth, the evaluation is intentionally limited. Material-only search can prefer unrealistic quiet moves or fluctuate between odd and even depths. A correct score under this model does not validate tactical quality. All implementations share the same move library, so agreement can miss a common rule or evaluation defect. The archive checks score agreement; stronger validation would independently evaluate selected best moves and add perft move-count checks without changing the measured algorithm.

Finally, the first Kiwipete one-worker split-three trial exceeded a 180-second limit; a later job repeated the same workload successfully three times with an 800-second allowance. An endgame pilot also required retrying. These attempts are preserved and excluded from successful medians. Retry policy and shallow fallback workload selection can bias what is easy to run, so the two short middlegames remain visible rather than disappearing from the report.

7.2 Recommendations tied to observed evidence

The measurements motivate testing decomposition as a function of workload and available workers; they do not validate an automatic selection rule. The 16-core comparisons motivate testing a shallower frontier before choosing split three. For large allocations, task size estimates or demand-driven distribution could address the measured rank imbalance, while bounded batches can control scheduling overhead. A coordinator consumes real resources and should be included in efficiency accounting.

Bound sharing is the next algorithmic priority: finding a useful child early and propagating safe bounds can reduce work inflation. It also introduces synchronization, stale bounds and cutoff cancellation, so a dynamic scheduler alone is not sufficient. Correctness must distinguish exact scores, lower bounds and upper bounds, including re-search when necessary. A sensible experiment would add one mechanism at a time and measure changed node counts, rank balance and total time under the same inputs.

For reducing the measured search region, the longer workloads show gains at high core counts; the wrapper results require a more qualified latency claim; for resource-sensitive use, the incremental time reduction may not justify the allocated core-time. The two short cases show no full-node advantage in this protocol; identifying a general size threshold requires a matched workload-size sweep. GPU redesign should first avoid CPU generation of branches that serial search would prune, then measure launch and device utilization separately. None of these proposed mechanisms is claimed as implemented here.

7.3 Artifact map and rebuilding

The source code and run instructions are publicly available at https://github.com/Erchiusx/dsa5208/tree/master/project3. The `project3/README.md` file explains how to build and run each backend. New CPU raw logs, per-trial records and summary tables are under `experiments/multiposition/`; `strong_scaling.csv` preserves both baseline definitions. Original CPU and GPU logs are in `experiments/raw/`, and original seeded-root diagnostics in `experiments/pv/`. The source revision and SHA-256 are stated in each archive; execution scripts and full FEN inputs allow reproducing each configuration on Vanda after loading the documented toolchain. No raw dataset download is required.

From the repository root, regenerate CPU summaries with:

```
python3 project3/scripts/summarize_multiposition.py
```

From `project3/`, run `make cpu` or `make mpi`; the README documents GPU compiler flags and PBS resource requests. NumPy and Matplotlib regenerate report plots and tables using `report/build_analysis.py`. Compile `report/main.tex` from that directory. The repository also contains the editable report source. Queue waiting is excluded from all performance measurements.

8 Conclusion and AI-use declaration

The experiments show that scaling a task graph and accelerating a search algorithm are different achievements. Deep frontiers fill many cores but increase the amount of search; a first-child bound preserves pruning but leaves load balance unresolved. Across the recorded new allocations, 720-core median search time is shorter than its serial reference on four longer positions and longer on two shallow ones. This is a descriptive comparison under the stated timing scope. The GPU experiments further show that reducing generated work helps, yet eliminating raw transfers would barely affect the measured depth-eight runtime. The causal contributions of cache locality, dynamic imbalance, collectives and GPU divergence remain unresolved. Useful optimization requires controlled changes and independent repeated allocations, not an attractive speedup curve alone.

The authors developed the algorithmic ideas, implementations, experimental design and Vanda measurements. OpenAI Codex assisted with preparing the figures and drafting and editing this report. The authors checked the figures, analysis and final text against the archived results and take responsibility for the submitted work. Third-party library use is credited separately above.

References

[1] D. E. Knuth and R. W. Moore. An analysis of alpha–beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975. https://doi.org/10.1016/0004-3702(75)90019-3.

[2] R. Feldmann, B. Monien, P. Mysliwietz and O. Vornberger. Distributed game-tree search. *ICCA Journal*, 12(2):65–73, 1989. https://doi.org/10.3233/ICG-1989-12203.

[3] Disservin. *chess-library*, MIT-licensed C++ library, revision `2a608d58f44776909bb6e2c09505e97ba1451319`. https://github.com/Disservin/chess-library.

[4] Open MPI. *mpirun/mpiexec documentation*, version 5.0.7. https://docs.open-mpi.org/en/v5.0.7/man-openmpi/man1/mpirun.1.html.

[5] OpenMP Architecture Review Board. *OpenMP API Specification 5.2*, 2021. https://www.openmp.org/specifications/.

A Exact benchmark positions

FEN encodes piece placement, side to move, castling and en-passant information, followed by move counters. Each string below is an exact input. The corresponding diagrams on the following pages show piece placement with White at the bottom; the labels state which side moves first.

Initial position (depth 8)

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

Kiwipete (depth 6)

```
r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/PPPBBPPP/R3K2R
w KQkq - 0 1
```

Rook/pawn (depth 9)

```
8/2p5/3p4/KP5r/1R3p1k/8/4P1P1/8 w - - 0 1
```

Tactical (depth 6)

```
r3k2r/Pppp1ppp/1b3nbN/nP6/BBP1P3/q4N2/Pp1P2PP/R2Q1RK1
w kq - 0 1
```

Minor pieces (depth 9)

```
8/5pk1/3b2p1/3Pp3/4P3/2N3P1/5PK1/8 w - - 0 1
```

Ruy Lopez (depth 4)

```
r1bq1rk1/2p1bppp/p1np1n2/1p2p3/4P3/1BP2N1P/PP1P1PP1/RNBQR1K1
b - - 0 9
```

Queen's Gambit (depth 4)

```
rn1q1rk1/pbp1bpp1/1p3n1p/3p4/3P3B/2NBPN2/PP3PPP/R2Q1RK1
b - - 3 10
```

A.1 Board diagrams

These diagrams visualize the seven benchmark inputs listed above. The search depth is measured in plies, or individual moves by one side.

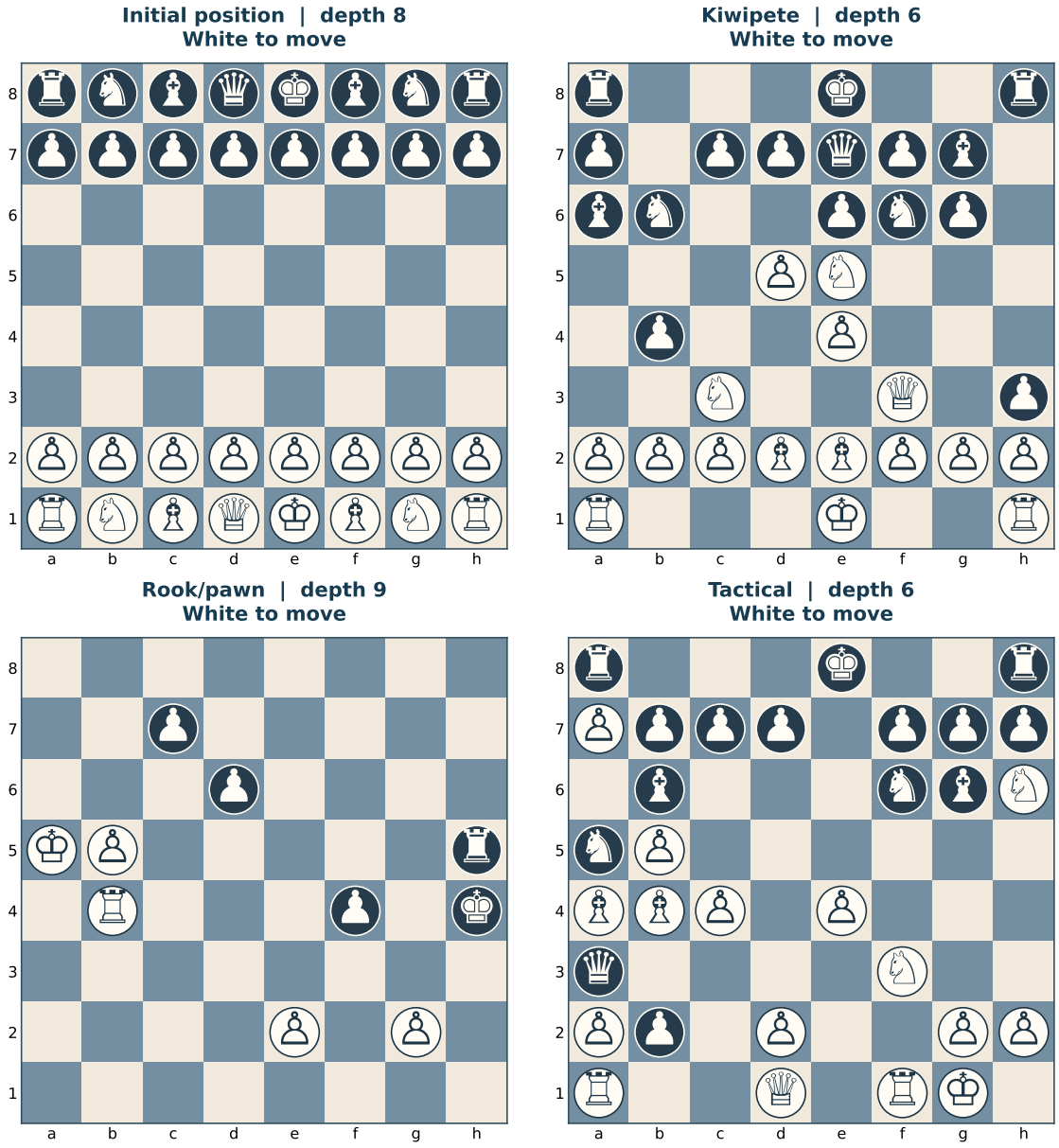


Figure 6: Initial position, Kiwipete, rook/pawn endgame and tactical position. Every board uses White's perspective; the label gives the side to move.

Board diagrams (continued)

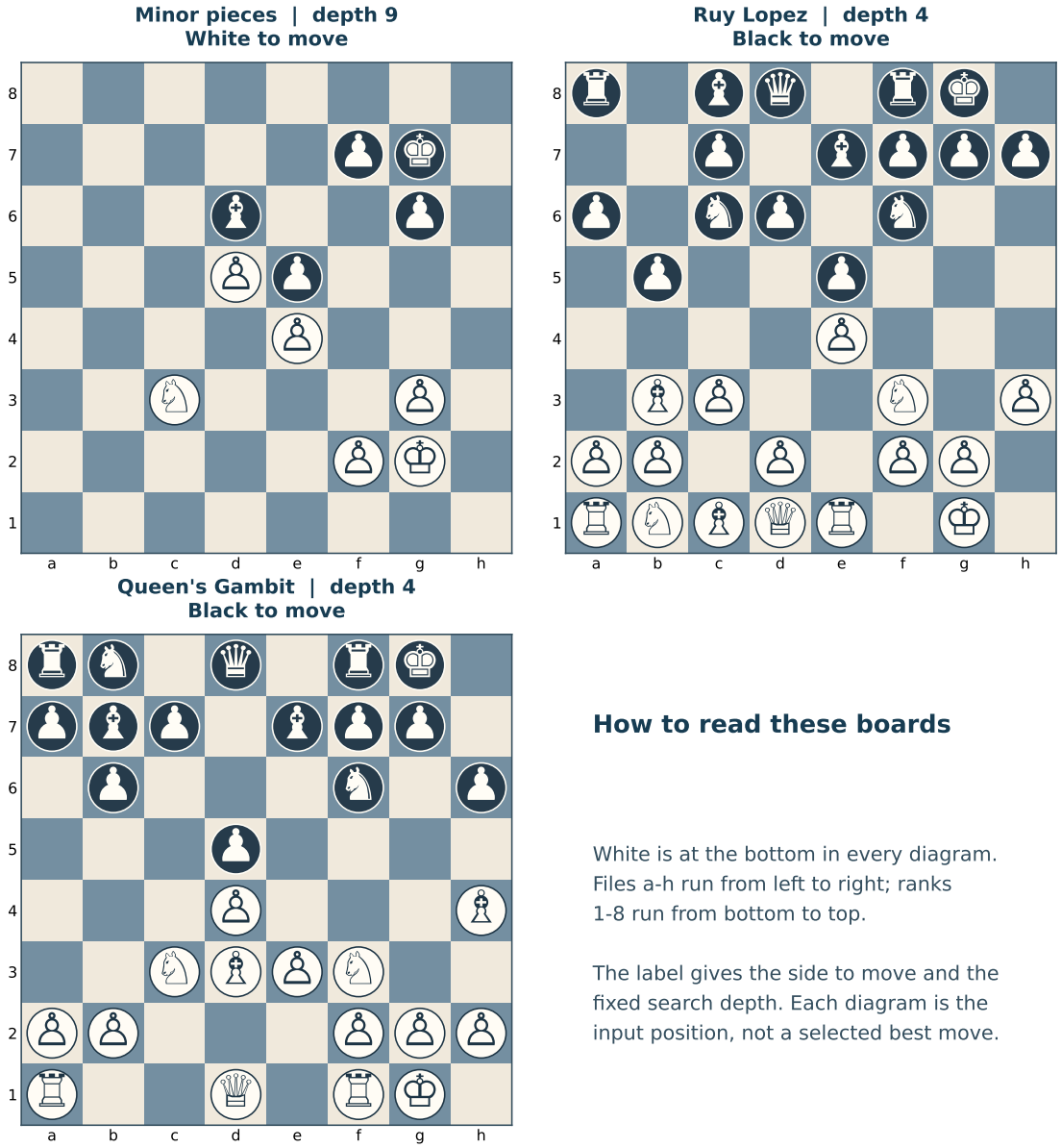


Figure 7: Minor-piece endgame, Ruy Lopez and Queen's Gambit positions. These are input positions, not moves recommended by the search.